Hands-On Generative AI

Retrieval-Augmented Generation

Day 3 - Session 3

KavinSchool

What We'll Cover

1. What RAG is and the two pipelines
2. Chunking and embedding model choice
3. Demo: chunking strategies
4. Vector databases: FAISS and Chroma
5. Agentic RAG
6. RAG evaluation and observability
7. Lab 3.4: end-to-end RAG

The Problem: Where LLMs Fail

Why we need RAG

KavinSchool

The Problem: Hallucinations

The model confidently states **false information** as fact.

```
Prompt: "What paper did van Rossum publish in Nature in 2019?"
Output: "Van Rossum published 'Dynamic Type Systems at Scale'
         in Nature, Volume 573..."  ❌ (completely fabricated)
```

> **Rule:** Never trust LLM output for factual claims on private or recent data without verification.

The Problem: Knowledge Cutoff & Context

Knowledge Cutoff

Models have a training cutoff date. They don't know what happened yesterday, and they don't know your company's internal wiki.

Long-Context Distraction

Even with 128K+ context windows, models can **lose information** in the middle of long inputs. ("Lost in the middle" problem).

We need a way to give models *exact, relevant* context right when they need it.

What RAG Is

Ground answers in retrieved text

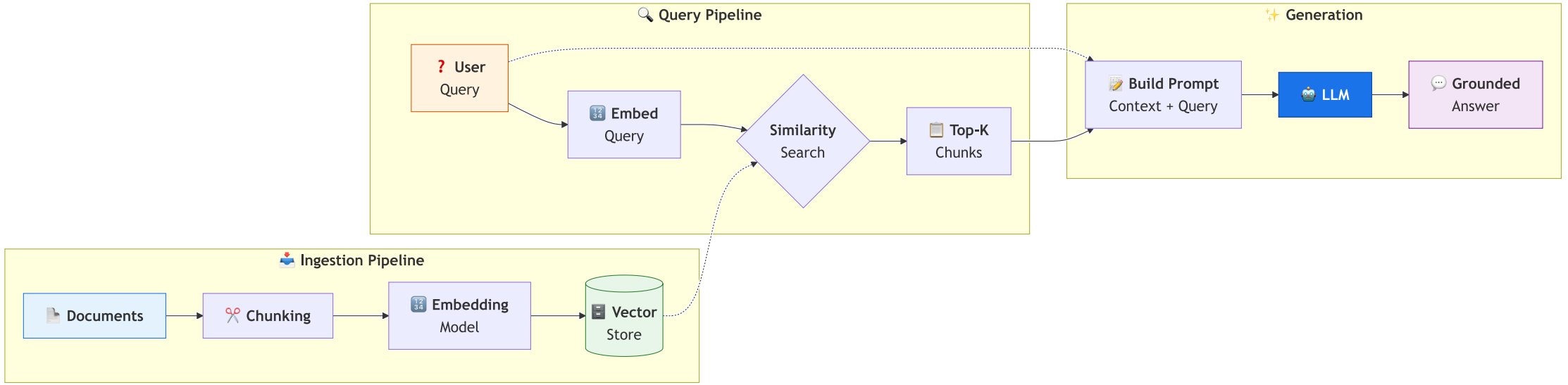
KavinSchool

The RAG Idea

RAG fetches relevant text at query time and puts it in the prompt. The model answers from that context instead of its training data alone.

- Reduces hallucination on private or changing facts
- Lets the model cite its sources
- Updates by changing the documents, not the model

The RAG Pipeline



Two Pipelines

RAG has an offline step and an online step.

- **Ingestion:** documents to chunks to embeddings to a vector store
- **Query:** embed the question, search, retrieve top-k, build the prompt, generate

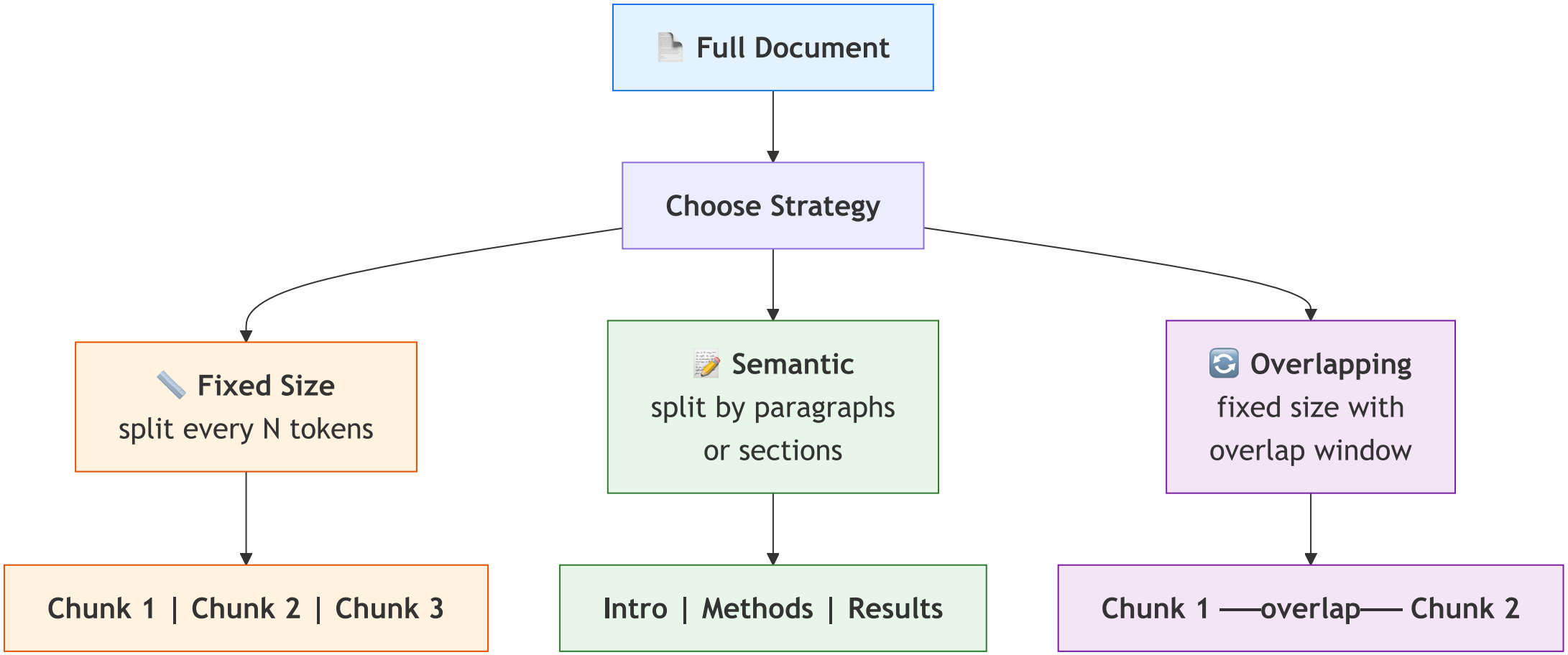
Ingestion runs once or on updates. The query pipeline runs on every question.

Chunking and Embeddings

Preparing documents for retrieval

KavinSchool

Chunking Strategies



Chunking Guidelines

Chunk size affects retrieval quality. Aim for pieces that hold one idea.

- Target 200 to 500 tokens per chunk
- Use 10 to 20 percent overlap to avoid mid-sentence cuts
- Split on paragraphs, sections, or headers
- Store metadata: source, section, chunk id

> **TIP**
>
> Always preserve metadata (like document title and section) when chunking. You can prepend this metadata to the chunk text before embedding so the model has context when it retrieves an isolated paragraph.

Demo: Chunking Strategies

Run `day3/demos/demo-chunking.py`.

- Chunk one document three ways: fixed, fixed with overlap, by paragraph
- Compare chunk counts and where each strategy cuts
- See how overlap keeps an idea from being split at a boundary

> Overlap repeats text so ideas are not cut off; paragraphs vary in size.

Choosing an Embedding Model

Pick by retrieval quality, dimension, cost, and whether it runs locally. For offline work, a local `sentence-transformers` model works well.

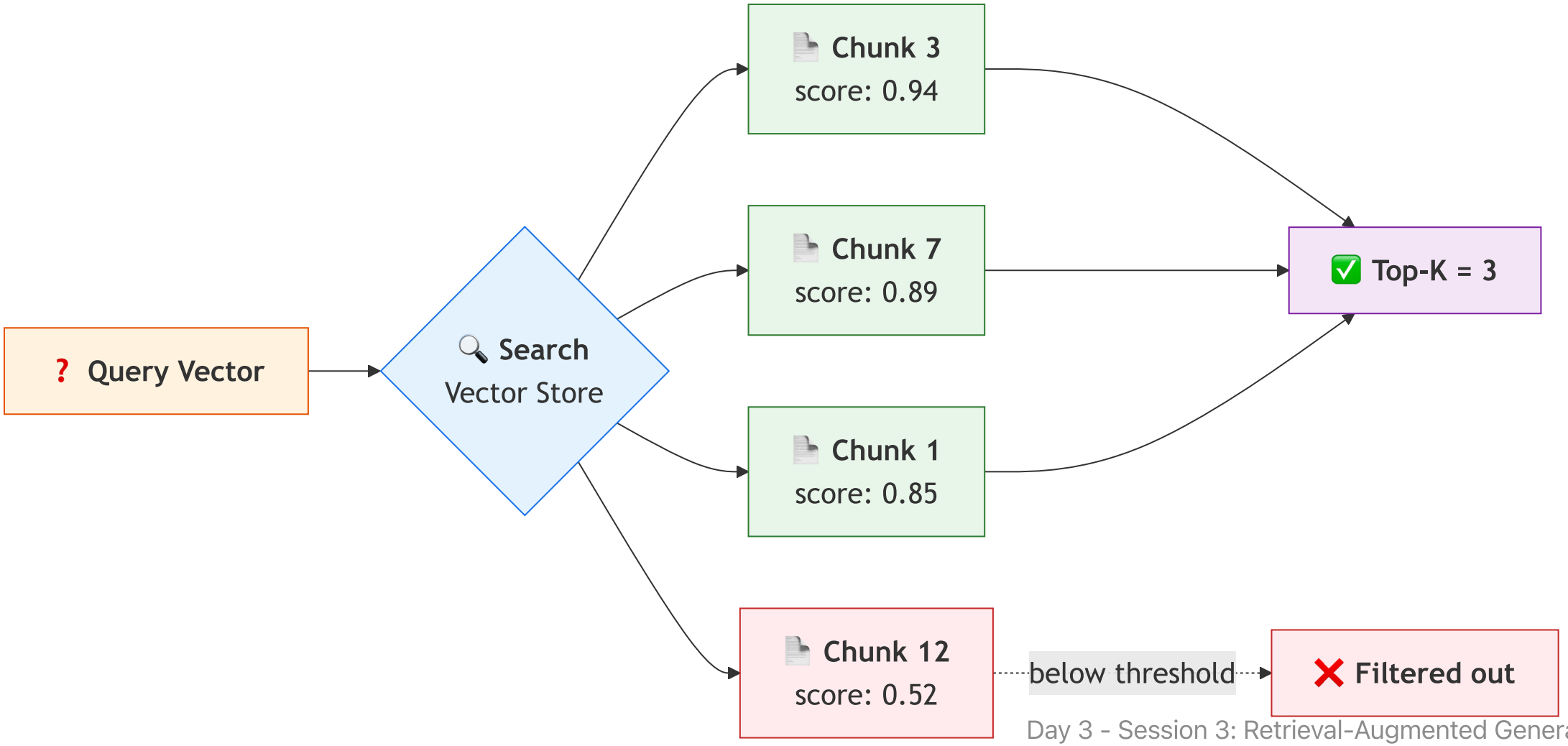
- Use the same model for ingestion and queries
- Larger models retrieve better at higher cost and latency
- `all-MiniLM-L6-v2` is a small, fast default

Vector Databases

FAISS and Chroma, local

KavinSchool

Vector Search



FAISS and Chroma

Both run locally. Pick based on what you need.

Store	Type	Best for
FAISS	In-memory library	Prototypes, small corpora
Chroma	Embedded database	Local apps, metadata, persistence

FAISS is fastest to stand up; Chroma stores documents and metadata together.

FAISS: Facebook AI Similarity Search. Chroma is an open-source embedding database.

> **IN PRACTICE**
>
> Enterprise teams often move from local FAISS prototypes to managed vector databases (Pinecone, Weaviate, or pgvector) when they need to scale across millions of documents or handle real-time updates.

FAISS in Code

```
import faiss
import numpy as np

index = faiss.IndexFlatL2(384)          # match embedding dimension
index.add(np.array(vectors, dtype=np.float32))

q = np.array([query_vec], dtype=np.float32)
distances, indices = index.search(q, k=3)
```

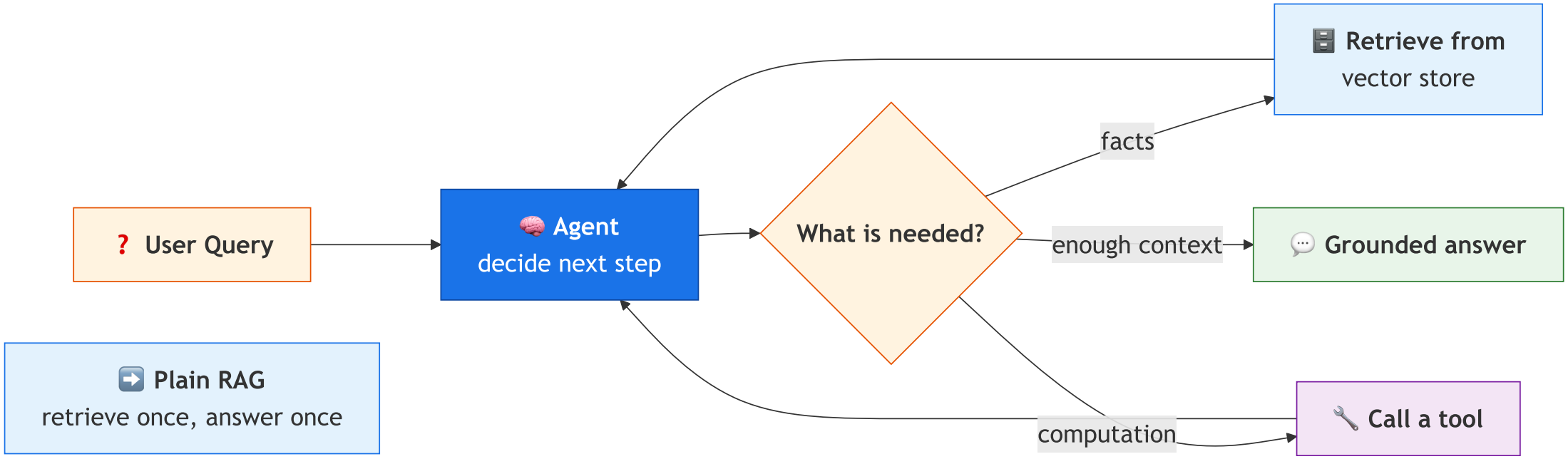
Add a score threshold to drop weak matches below the top-k.

Agentic RAG

Let the model decide when to retrieve

KavinSchool

Plain RAG vs Agentic RAG



When Agentic RAG Helps

Plain RAG retrieves once and answers once. An agent decides each step.

- Retrieve again with a refined query if the first pass is weak
- Call a tool for computation the documents cannot provide
- Answer once it has enough grounded context

Use it when a single retrieval is not enough to answer well.

Evaluating RAG

Know what to fix

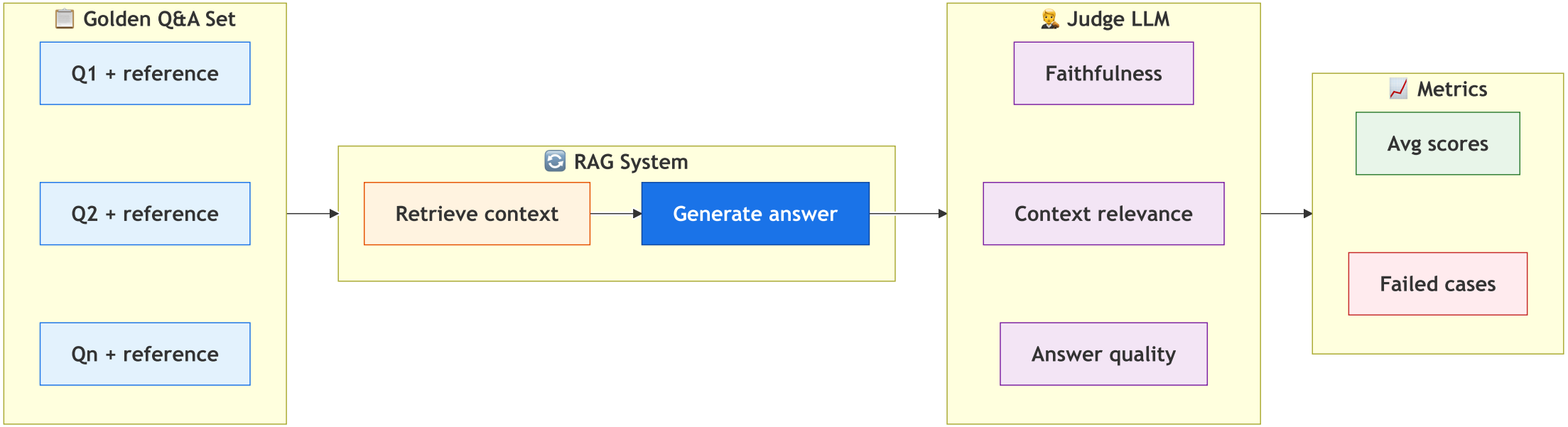
KavinSchool

What Can Go Wrong

RAG can fail at retrieval or at generation. Evaluation separates the two.

- Wrong chunks retrieved: fix chunking or the embedding model
- Right chunks, wrong answer: fix the prompt or generation
- Track a small set of metrics over a fixed test set

LLM-as-a-Judge



RAG Metrics

Metric	Measures	Low score means
Recall@k	Correct chunk in top-k	Retriever misses text
Faithfulness	Claims supported by context	Model hallucinating
Context relevance	Context on-topic	Noisy retrieval
Answer quality	Clear, complete answer	Prompt needs work

Keep the judge at temperature 0.0 for stable scores.

> **WARNING**
>
> LLM judges exhibit position bias, where they favor the first answer they read. Swap the order of answers in your evaluation prompt to check for bias.

Observability

Log enough to debug a bad answer without guessing.

- The query and retrieved chunk ids with scores
- The final prompt sent to the model
- The answer and judge scores

Aggregate scores over time to catch regressions after a change.

Intuit Scenario: TurboTax Document QA

RAG securely answers user questions based on large regulatory documents.

- **Base model:** General-purpose LLM (GPT-4 class)
- **Task:** Answer user tax questions accurately without hallucinating rules
- **Dataset:** IRS tax code PDFs, chunked and embedded in a vector store
- **Outcome:** The model retrieves the exact tax code section and cites it, ensuring compliance and building user trust

Breakout Lab 3.4

End-to-end RAG

KavinSchool

Lab 3.4: Build a RAG System

Open `day3/lab3_4_rag/start.ipynb` in the companion repo.

- Chunk and embed a small corpus with sentence-transformers
- Index in FAISS or Chroma and retrieve top-k
- Generate through `shared/llm_client.py` at tier `mini`
- Run a small LLM-as-judge eval over a few questions

> Stretch goal: add an agentic RAG tool call that computes an answer.

Key Takeaways

- [x] RAG grounds answers in retrieved text to reduce hallucination
- [x] Ingestion builds the store; the query pipeline runs per question
- [x] Chunking and embedding choice drive retrieval quality
- [x] FAISS and Chroma both run locally for small corpora
- [x] Evaluate with recall@k, faithfulness, and an LLM judge

Questions?

Next: Deployment Optimization and Capstone

Short break, then Session 4

KavinSchool